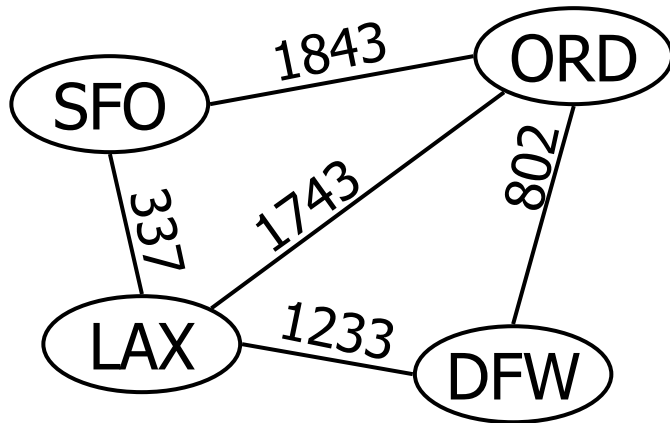


Lecture 08b Graph Algorithms

Learning Outcome

- Depth-first search
- Breadth-First Search
- Topological Sorting

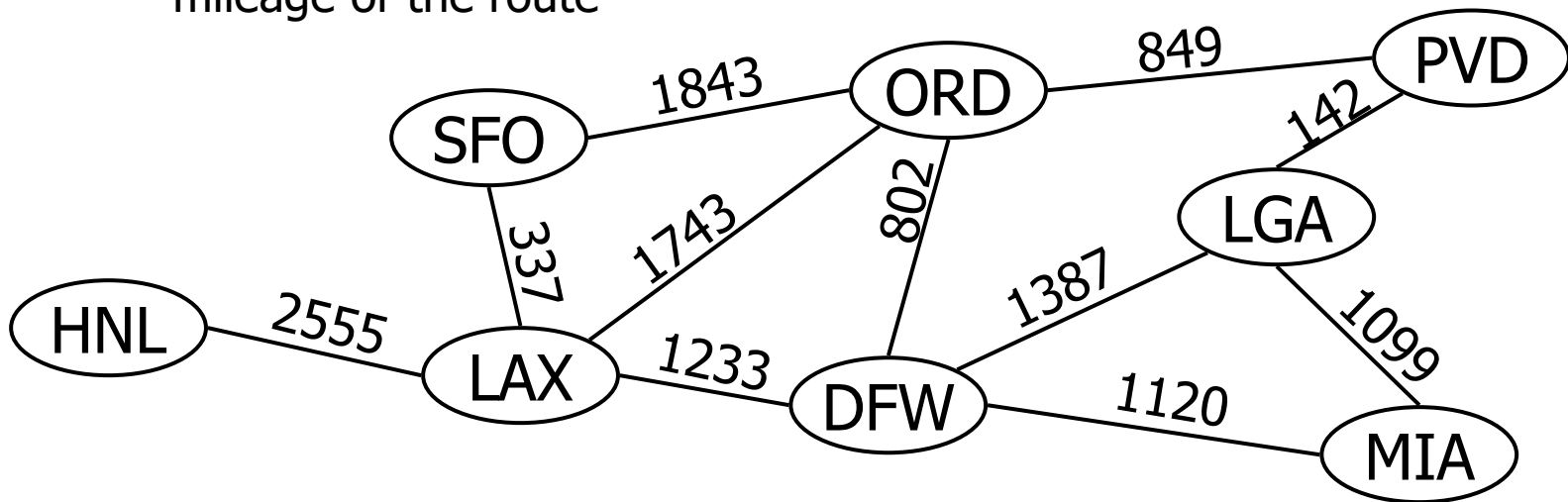


Graph

- ◆ A graph is a pair (V, E) , where
 - V is a set of nodes, called vertices
 - E is a collection of pairs of vertices, called edges
 - Vertices and edges are positions and store elements

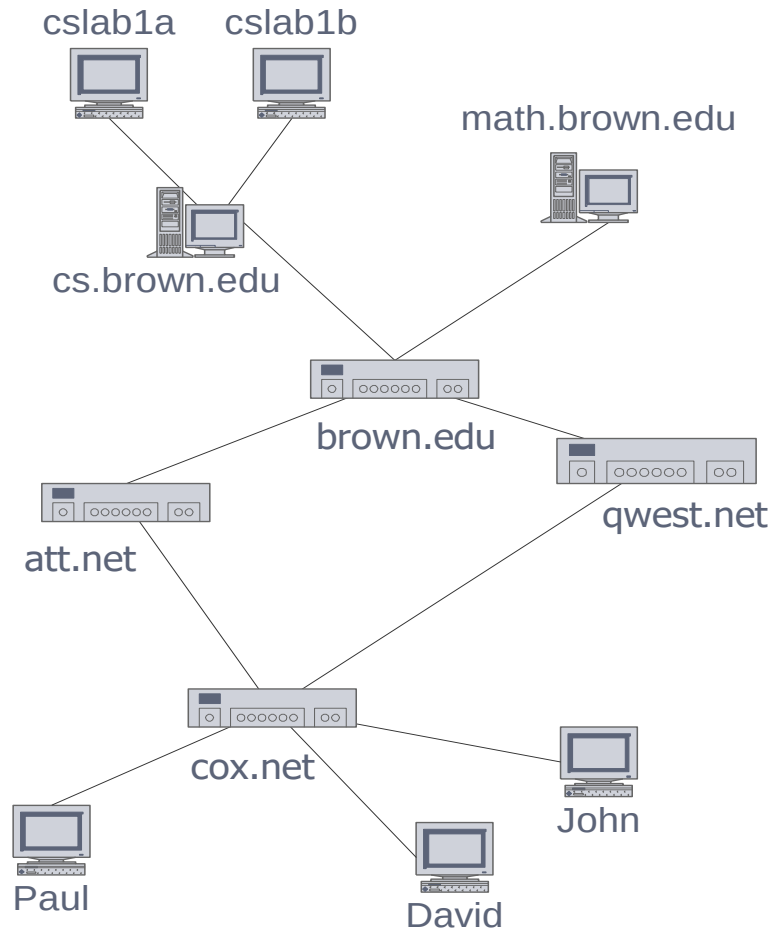
- ◆ Example:

- A vertex represents an airport and stores the three-letter airport code
- An edge represents a flight route between two airports and stores the mileage of the route



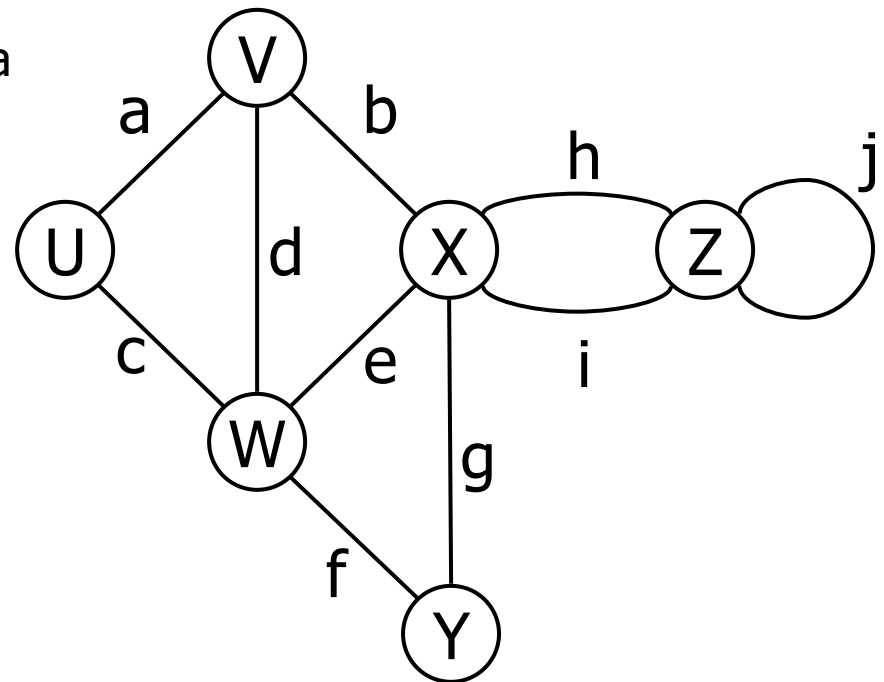
Applications

- ◆ Electronic circuits
 - Printed circuit board
 - Integrated circuit
- ◆ Transportation networks
 - Highway network
 - Flight network
- ◆ Computer networks
 - Local area network
 - Internet
 - Web
- ◆ Databases
 - Entity-relationship diagram



Terminology

- ◆ End vertices (or endpoints) of an edge
 - U and V are the endpoints of a
- ◆ Edges incident on a vertex
 - a, d, and b are incident on V
- ◆ Adjacent vertices
 - U and V are adjacent
- ◆ Degree of a vertex
 - X has degree 5
- ◆ Parallel edges
 - h and i are parallel edges
- ◆ Self-loop
 - j is a self-loop



Properties

Property 1

$$\sum_v \deg(v) = 2m$$

Proof: each edge is counted twice

Notation

n	number of vertices
m	number of edges
$\deg(v)$	degree of vertex v

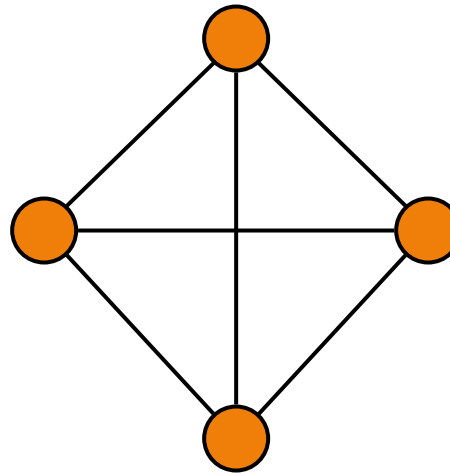
Property 2

In an undirected graph with no self-loops and no multiple edges

$$m \leq n(n-1)/2$$

Proof: each vertex has degree at most $(n-1)$

What is the bound for a directed graph?



Example

- $n = 4$
- $m = 6$
- $\deg(v) = 3$

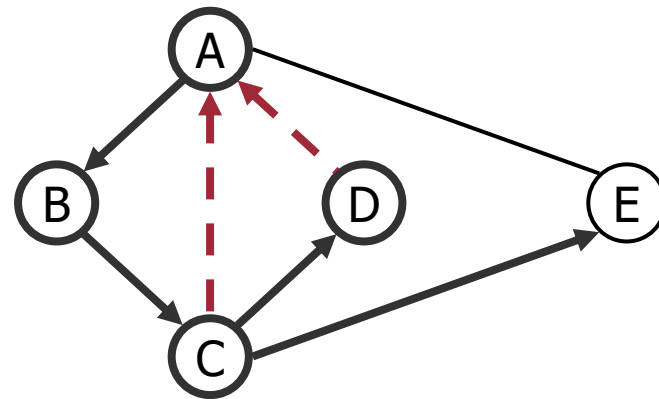
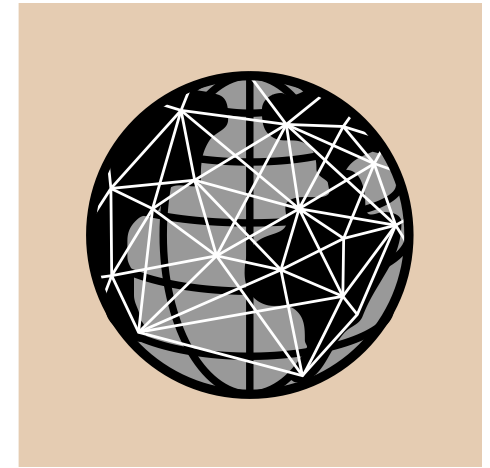
Asymptotic Performance

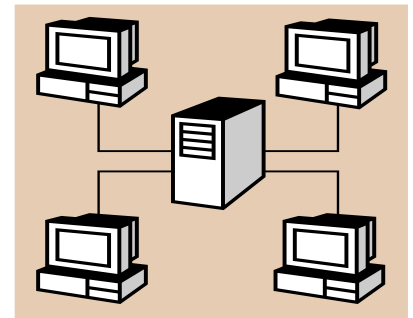
◆ n vertices, m edges ◆ no parallel edges ◆ no self-loops ◆ Bounds are “big-Oh”	Adjacency List	Adjacency Matrix
Space	$n + m$	n^2
incidentEdges(v)	deg(v)	n
areAdjacent(v , w)	min(deg(v), deg(w))	1
insertVertex(o)	1	n^2
insertEdge(v , w , o)	1	1
removeVertex(v)	$n + m$	n^2
removeEdge(v , w)	max(deg(v), deg(w))	1

Depth-First Search:

◆ Depth-first search (6.3.1)

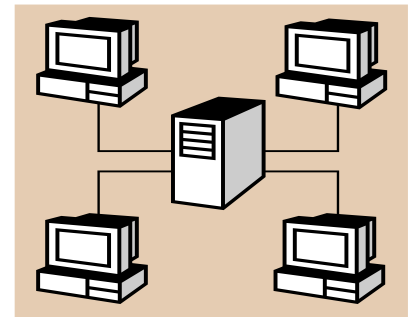
- Algorithm
- Example
- Properties
- Analysis





Depth-First Search

- ◆ Depth-first search (DFS) is a general technique for traversing a graph
- ◆ A DFS traversal of a graph G
 - Visits all the vertices and edges of G
 - Determines whether G is connected
 - Computes the connected components of G
 - Computes a spanning forest of G
- ◆ DFS on a graph with n vertices and m edges takes $O(n + m)$ time
- ◆ DFS can be further extended to solve other graph problems
 - Find and report a path between two given vertices
 - Find a cycle in the graph
- ◆ Depth-first search is to graphs what Euler tour is to binary trees



DFS Algorithm

- ◆ The algorithm uses a mechanism for setting and getting “labels” of vertices and edges

Algorithm DFS(G)

Input: graph G

Output: labeling of the edges of G
as discovery edges and back edges

for each u in $G.vertices()$

$u.label = UNEXPLORED$

for each e in $G.edges()$

$e.label = UNEXPLORED$

for each v in $G.vertices()$

 if $v.label == UNEXPLORED$

 DFS(G, v)

Algorithm DFS(G, v)

Input: graph G and a start vertex v

Output: labeling of the edges of G
in the connected component of v
as discovery edges and back edges

$v.label = VISITED$

for each e in $G.incidentEdges(v)$

 if $e.label == UNEXPLORED$

$w = opposite(v, e)$

 if $w.label == UNEXPLORED$

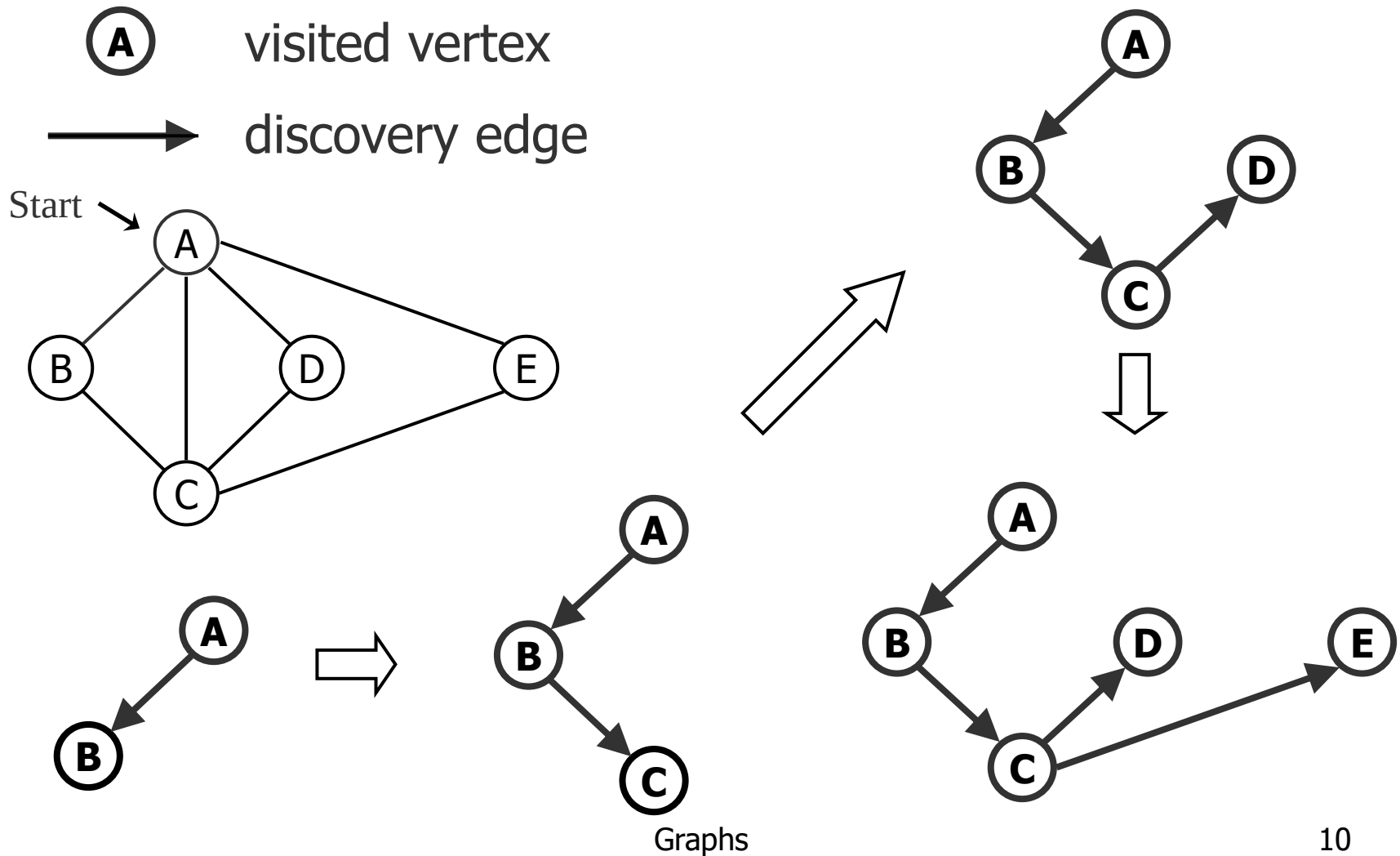
$e.label = DISCOVERY$

 DFS(G, w)

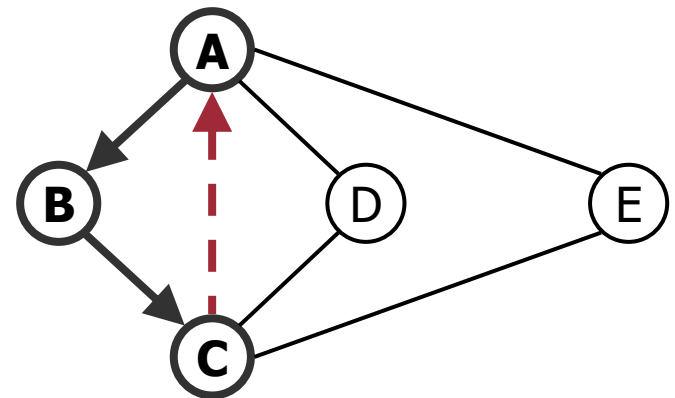
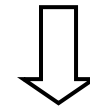
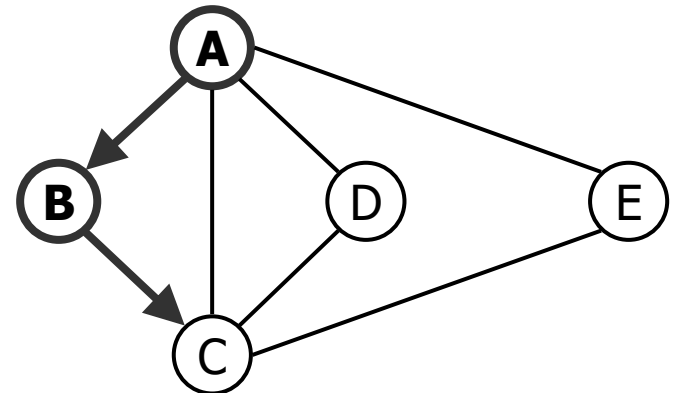
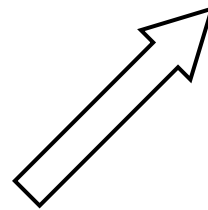
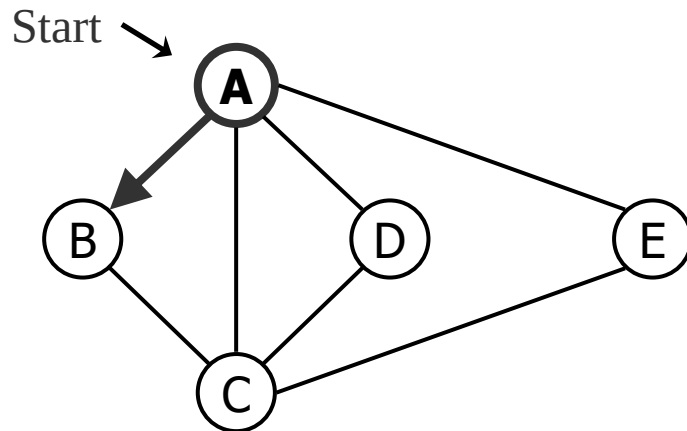
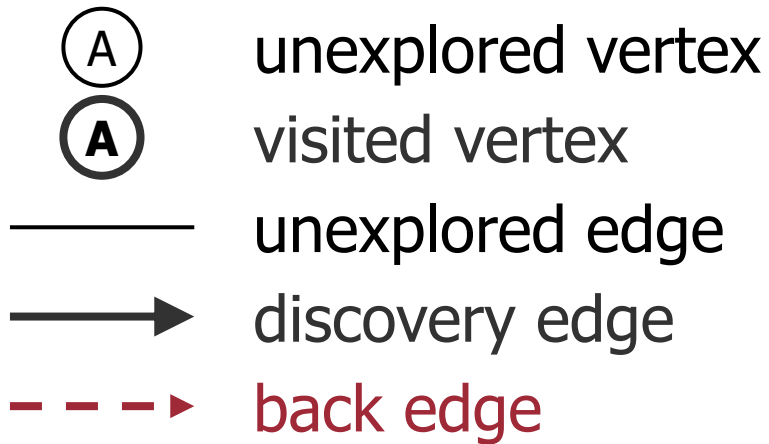
 else

$e.label = BACK$

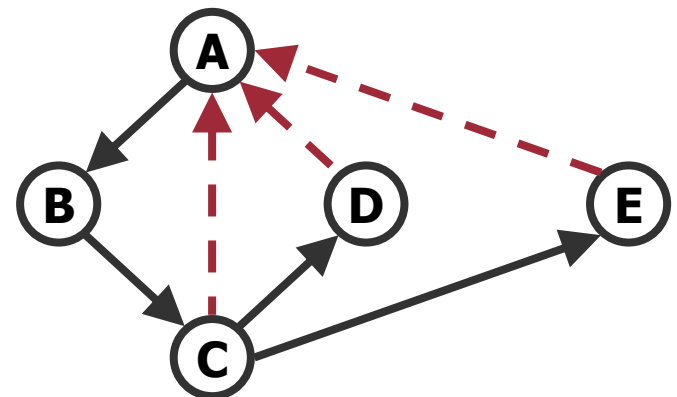
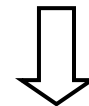
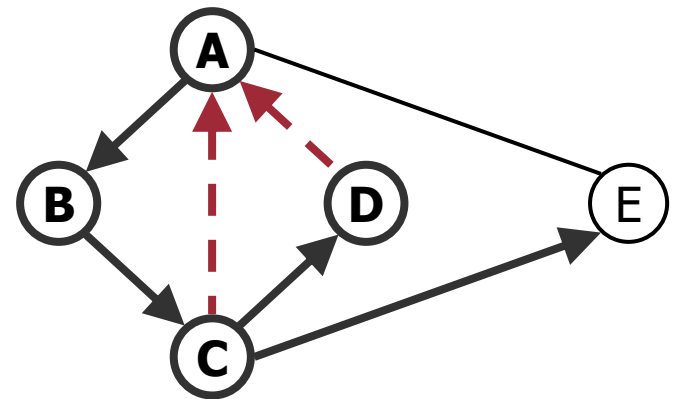
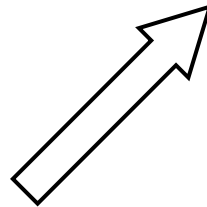
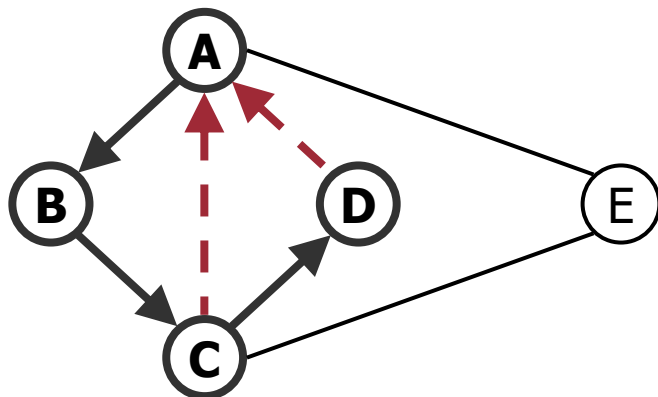
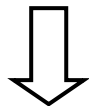
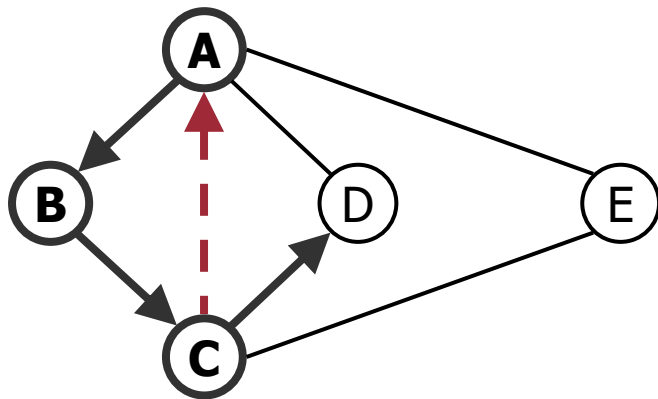
DFS – Simple Steps



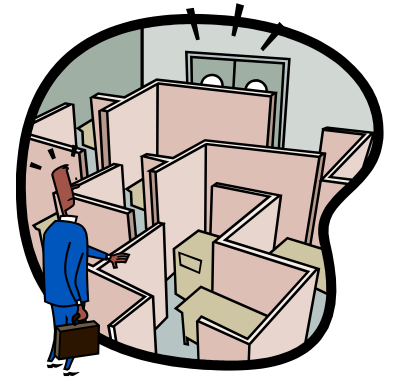
DFS - Detailed Steps



DFS - Detailed Steps (cont.)

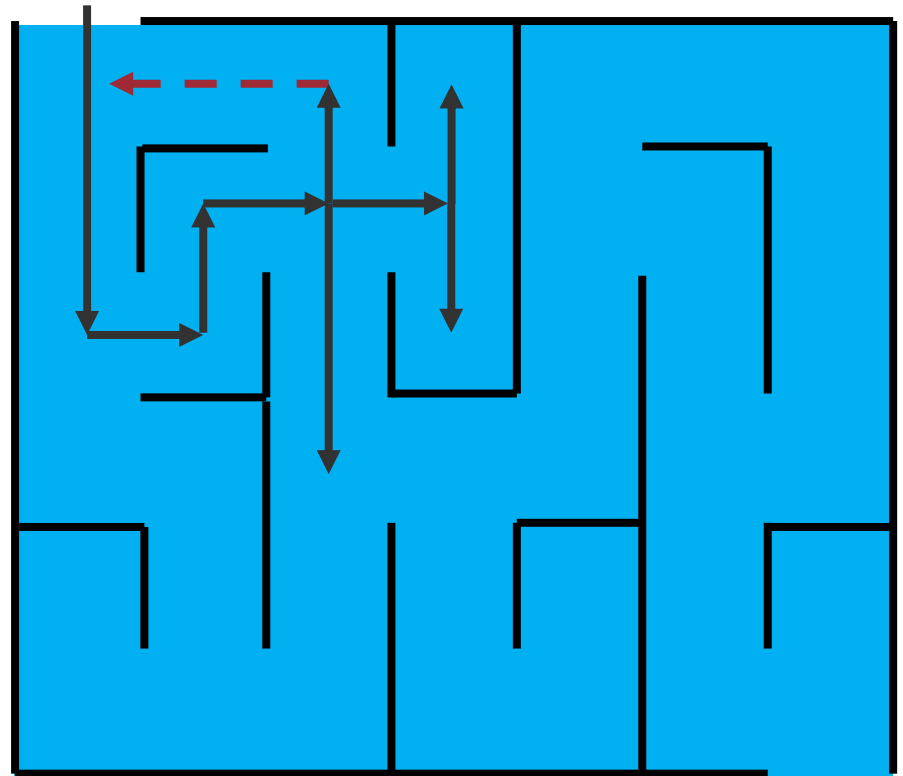


DFS and Maze Traversal



◆ The DFS algorithm is similar to a classic strategy for exploring a maze

- We mark each intersection, corner and dead end (vertex) visited
- We mark each corridor (edge) traversed
- We keep track of the path back to the entrance (start vertex) by means of a rope (recursion stack)



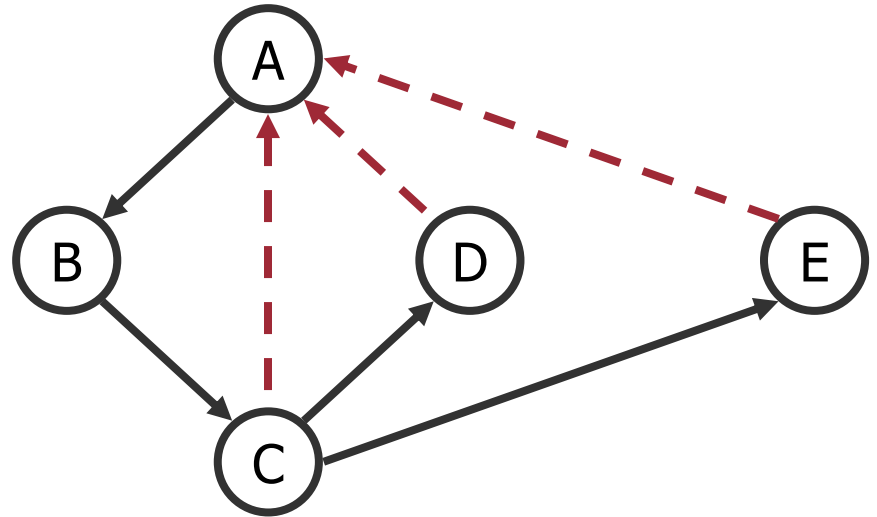
Properties of DFS

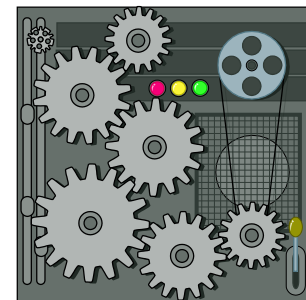
Property 1

$DFS(G, v)$ visits all the vertices and edges in the connected component of v

Property 2

The discovery edges labeled by $DFS(G, v)$ form a spanning tree of the connected component of v

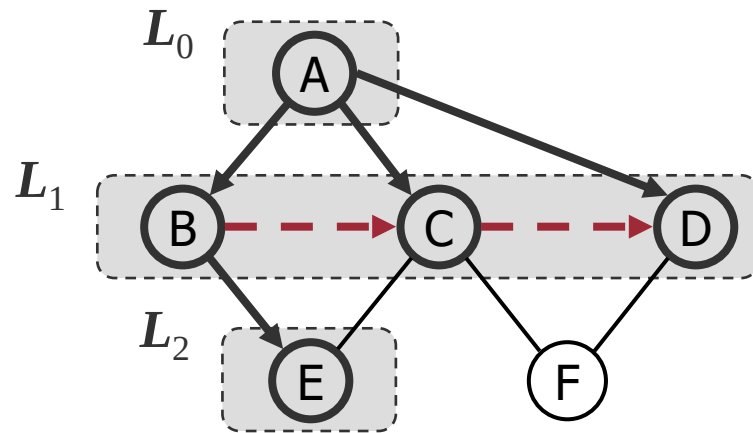




Analysis of DFS

- ◆ Setting/getting a vertex/edge label takes $O(1)$ time
- ◆ Each vertex is labeled twice
 - once as UNEXPLORED
 - once as VISITED
- ◆ Each edge is labeled twice
 - once as UNEXPLORED
 - once as DISCOVERY or **BACK**
- ◆ Method incidentEdges is called once for each vertex
- ◆ DFS runs in $O(n + m)$ time provided the graph is represented by the adjacency list structure
 - Recall that $\sum_v \deg(v) = 2m$

Breadth-First Search



Breadth-First Search

- ◆ Breadth-first search (BFS) is a general technique for traversing a graph
- ◆ A BFS traversal of a graph G
 - Visits all the vertices and edges of G
 - Determines whether G is connected
 - Computes the connected components of G
 - Computes a spanning forest of G
- ◆ BFS on a graph with n vertices and m edges takes $O(n + m)$ time
- ◆ BFS can be further extended to solve other graph problems
 - Find and report a path with the minimum number of edges between two given vertices
 - Find a simple cycle, if there is one

BFS Algorithm

- ◆ The algorithm uses a mechanism for setting and getting “labels” of vertices and edges

Algorithm BFS(G)

Input: graph G

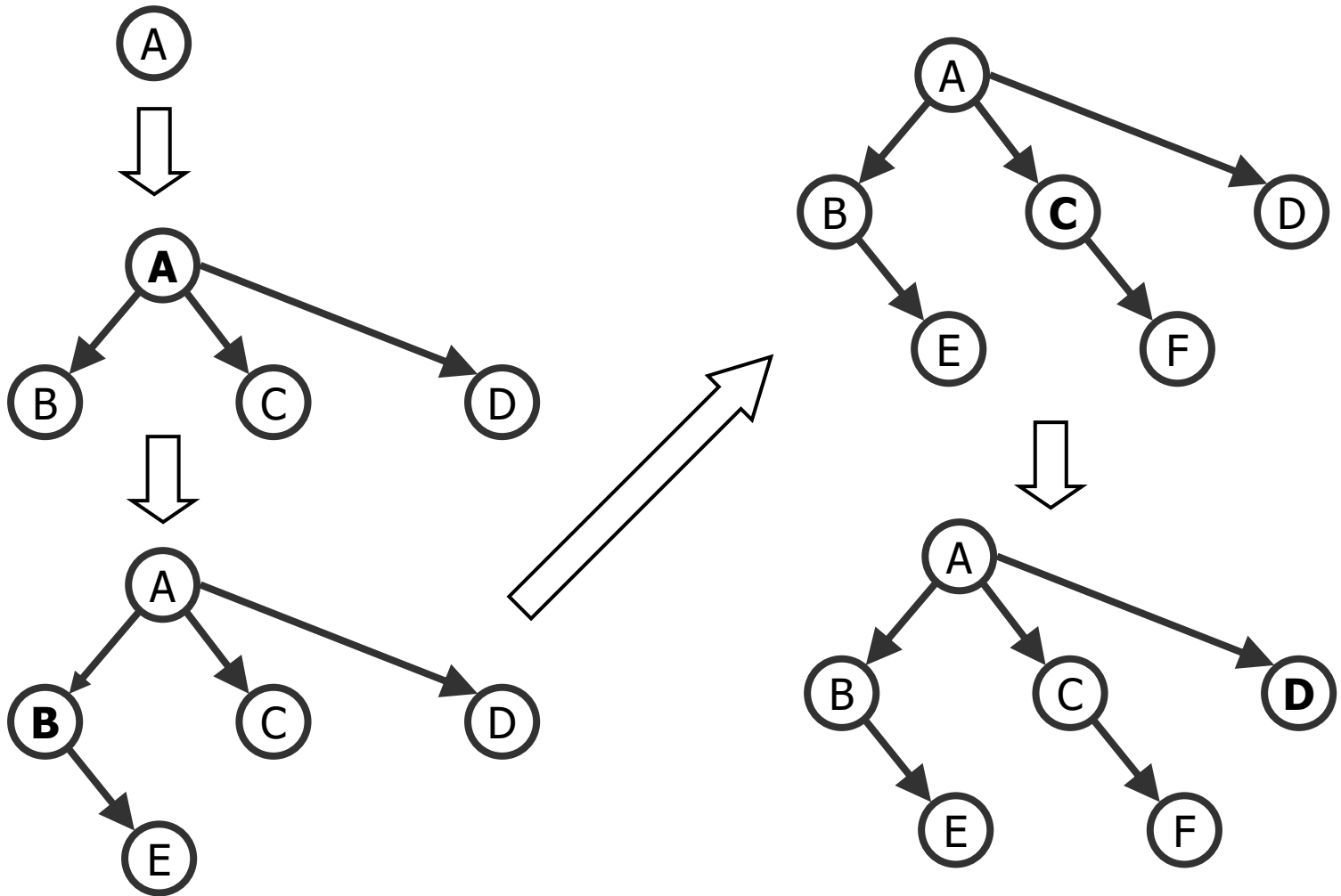
Output: labeling of the edges
and partition of the vertices
of G

```
for each u in G.vertices()
    u.label = UNEXPLORED
for each e in G.edges()
    e.label = UNEXPLORED
for each v in G.vertices()
    if v.label == UNEXPLORED
        BFS(G, v)
```

Algorithm BFS(G, s)

```
L0 = new empty sequence
L0.insertLast(s)
s.label = VISITED
i = 0
while not Li.isEmpty()
    Li+1 = new empty sequence
    for each v in Li.elements()
        for each e in G.incidentEdges(v)
            if e.label == UNEXPLORED
                w = opposite(v,e)
                if w.label = UNEXPLORED
                    e.label = DISCOVERY
                    w.label = VISITED
                    Li+1.insertLast(w)
            else
                e.label = CROSS
    i = i + 1
```

BFS – Simple Steps



BFS – Simple Steps

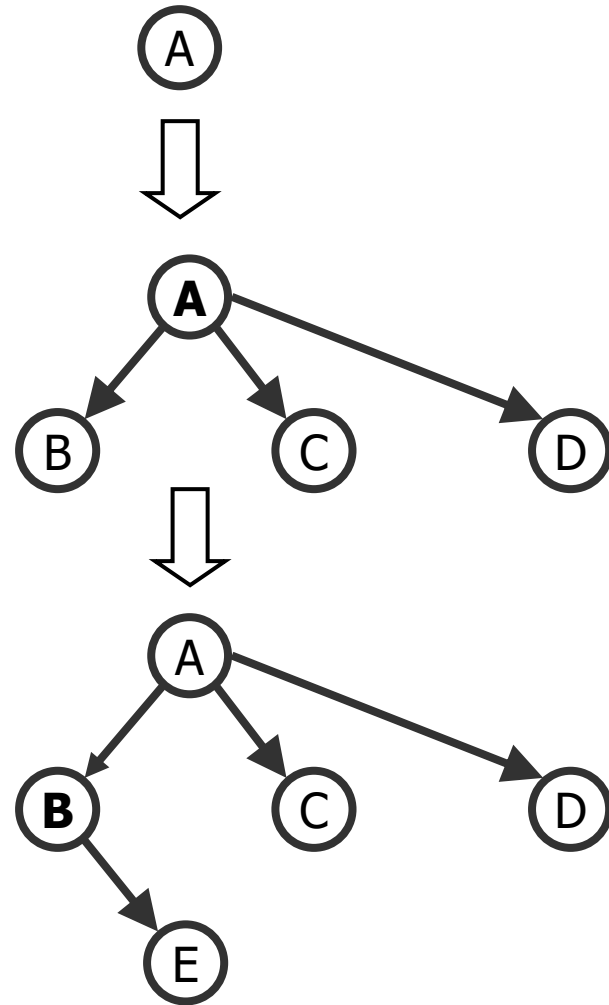
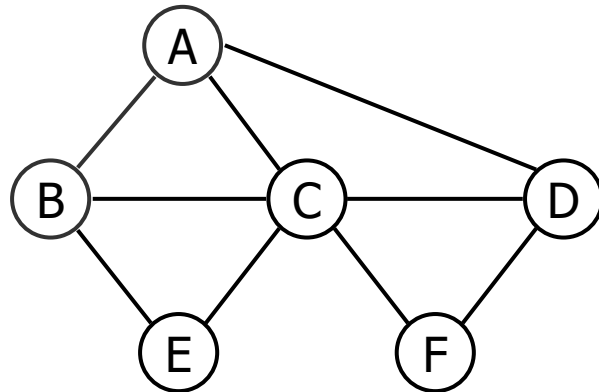
Ⓐ

visited vertex

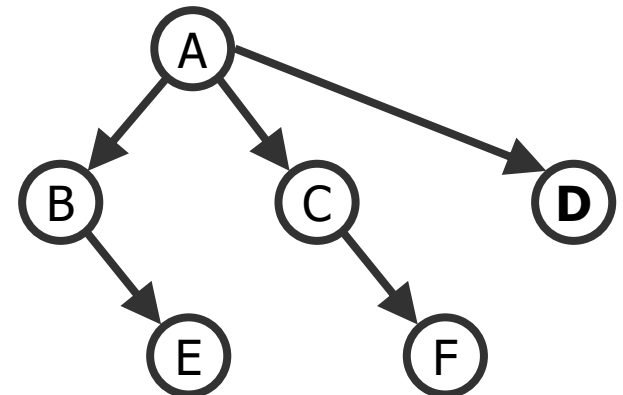
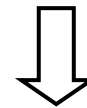
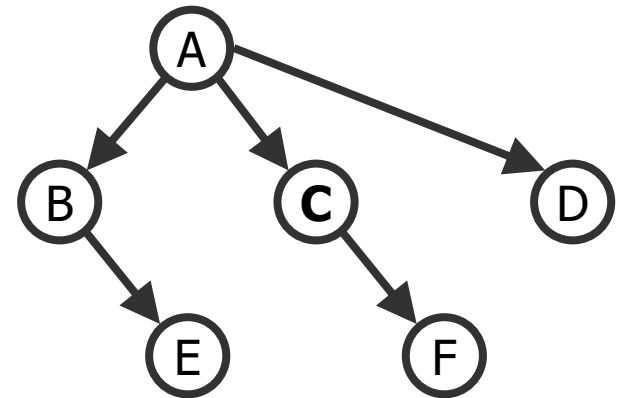
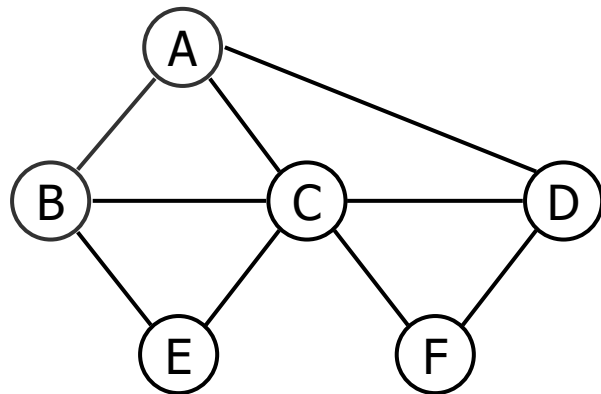


discovery edge

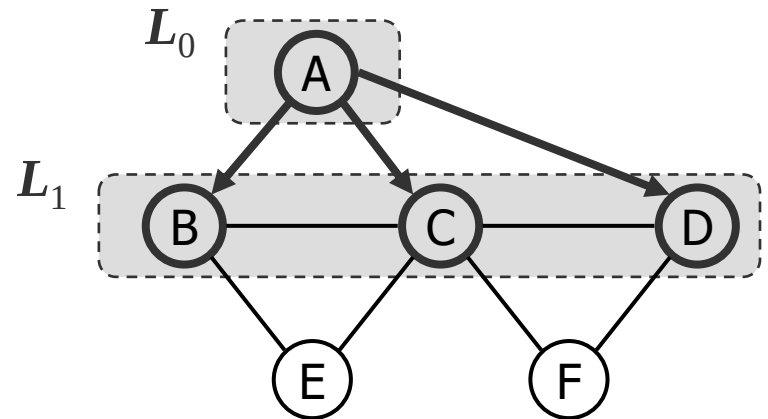
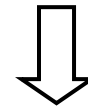
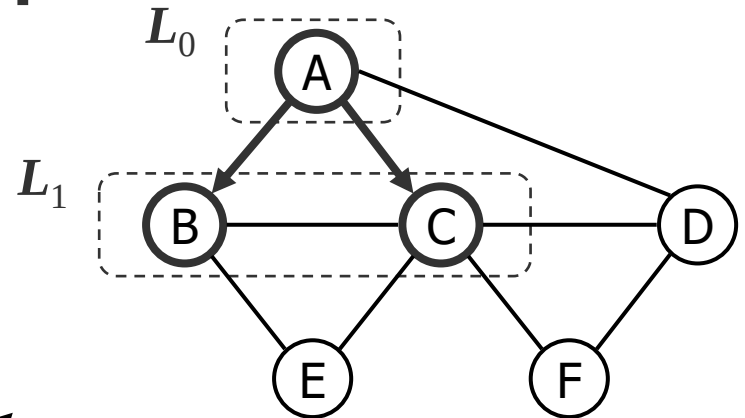
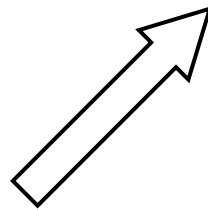
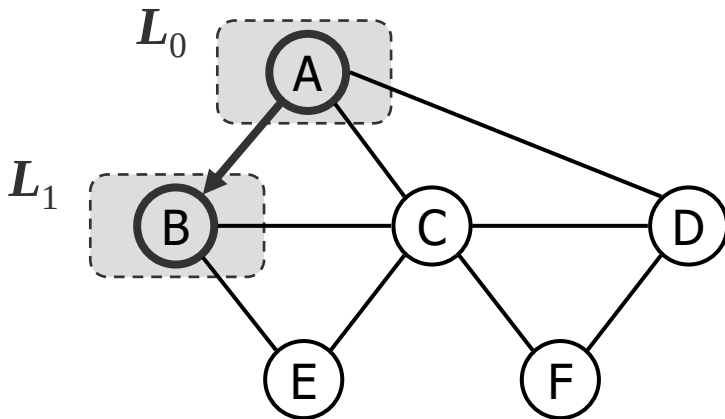
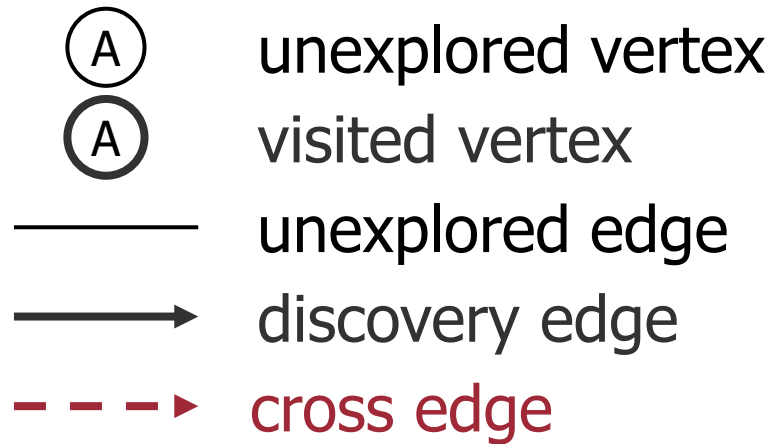
Start →



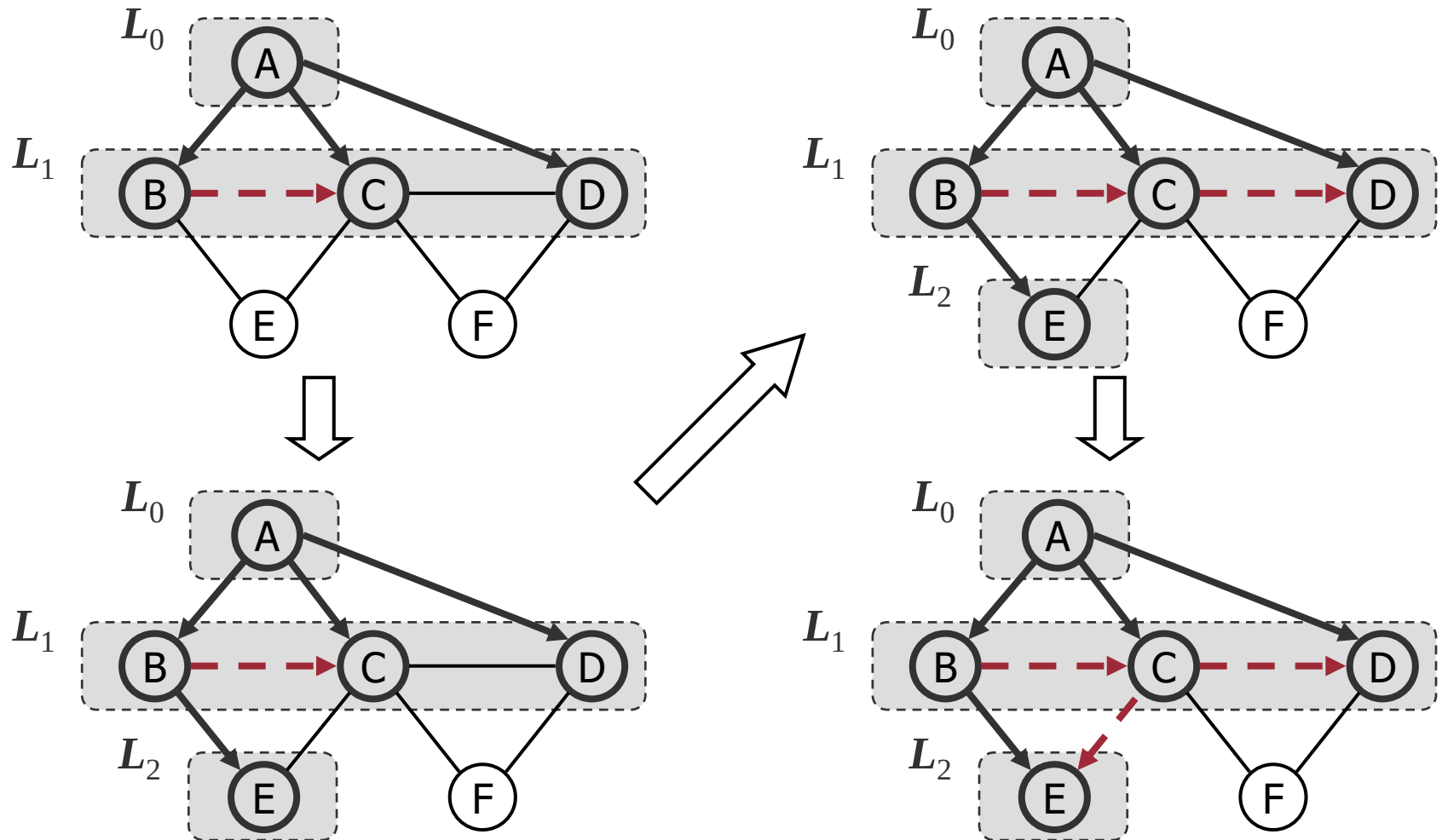
BFS – Simple Steps (cont.)



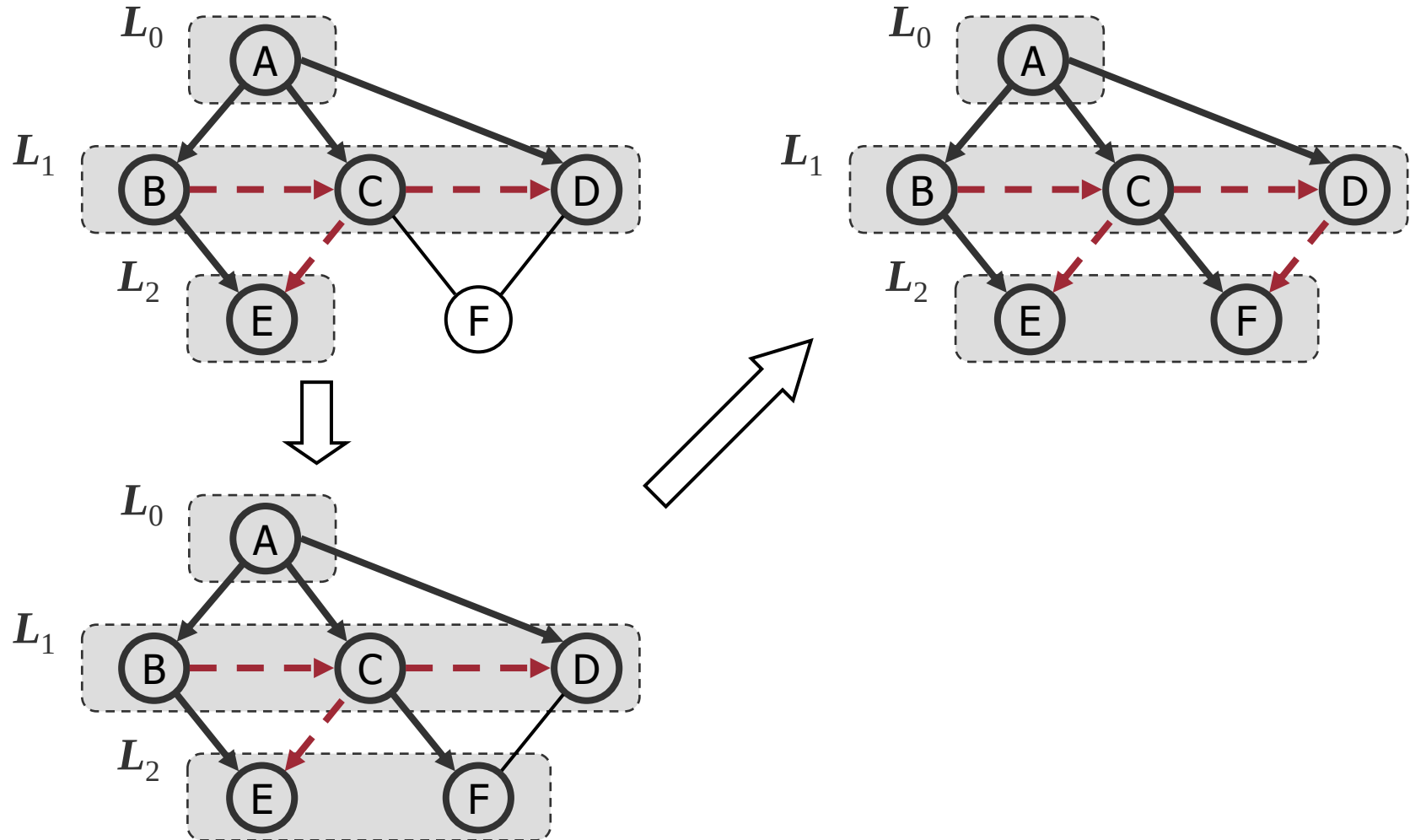
BFS – Detailed Steps



BFS – Detailed Steps (cont.)



BFS – Detailed Steps (cont.)



Properties

Notation

G_s : connected component of s

Property 1

$BFS(G, s)$ visits all the vertices and edges of G_s

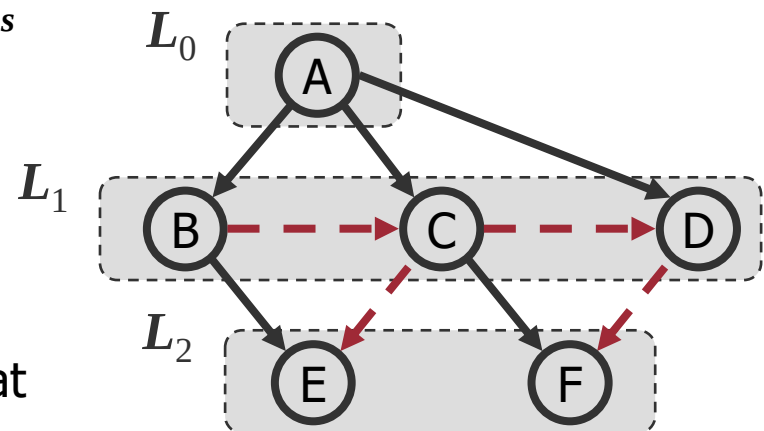
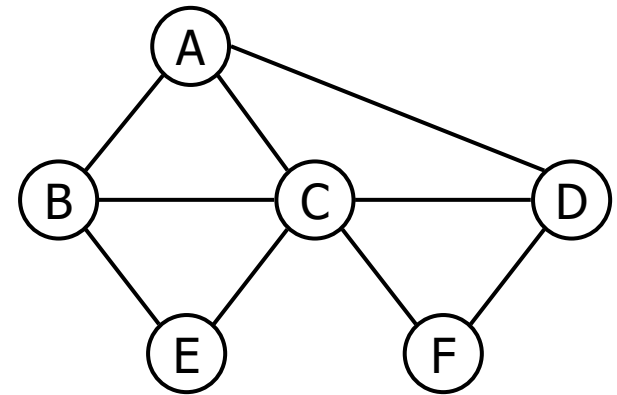
Property 2

The discovery edges labeled by $BFS(G, s)$ form a spanning tree T_s of G_s

Property 3

For each vertex v in L_i

- The path of T_s from s to v has i edges
- Every path from s to v in G_s has at least i edges



Analysis

- ◆ Setting/getting a vertex/edge label takes $O(1)$ time
- ◆ Each vertex is labeled twice
 - once as UNEXPLORED
 - once as VISITED
- ◆ Each edge is labeled twice
 - once as UNEXPLORED
 - once as DISCOVERY or **CROSS**
- ◆ Each vertex is inserted once into a sequence L_i
- ◆ Method incidentEdges is called once for each vertex
- ◆ BFS runs in $O(n + m)$ time provided the graph is represented by the adjacency list structure
 - Recall that $\sum_v \deg(v) = 2m$

DAGs and Topological Sorting

- ◆ A directed acyclic graph (DAG) is a digraph that has no directed cycles
- ◆ A topological sorting/ordering of a digraph is a numbering

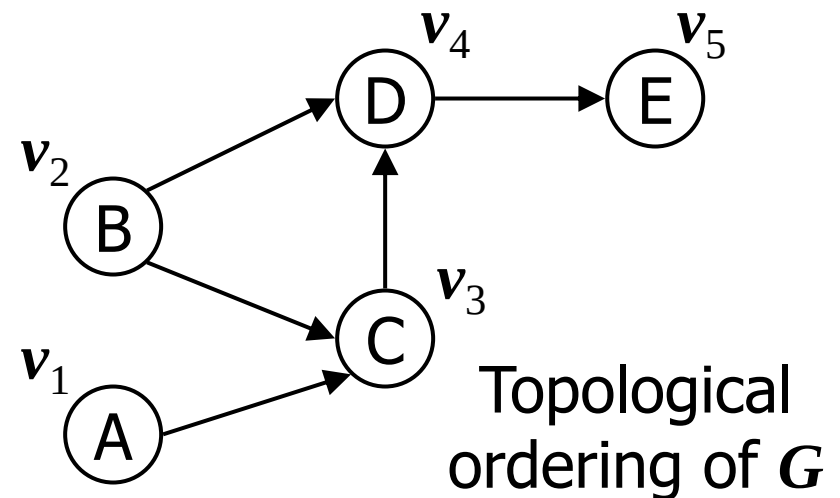
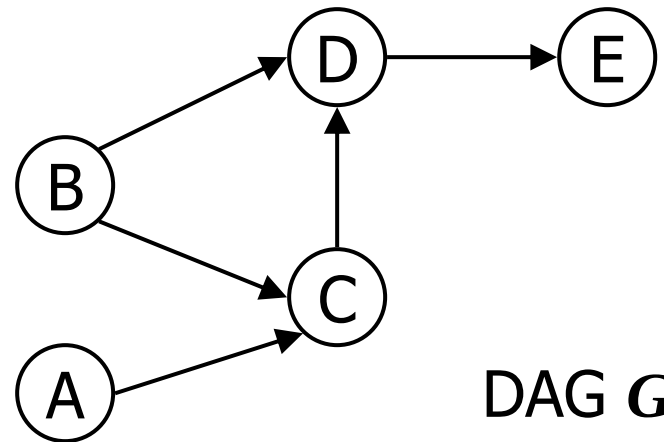
$$v_1, \dots, v_n$$

of the vertices such that for every edge (v_i, v_j) , we have $i < j$

- ◆ Example: in a task scheduling digraph, a topological sorting a task sequence that satisfies the precedence constraints

Theorem

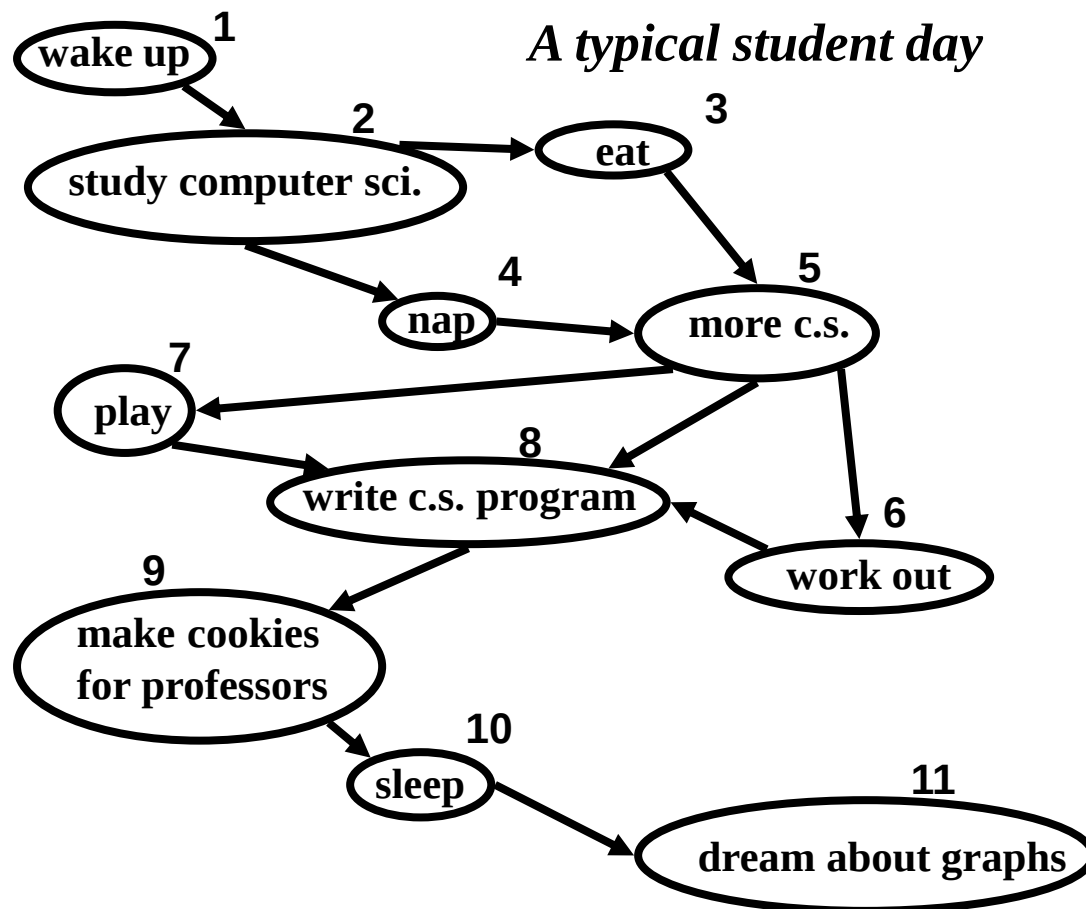
A digraph admits a topological sorting if and only if it is a DAG





Topological Sorting

- ◆ Number vertices, so that (u,v) in E implies $u < v$



Topological Sorting Algorithm 1

◆ Running time: $O(n + m)$.

```
Algorithm TopologicalSort(G)
  H = G          // Temporary copy of G
  n = G.numVertices()
  while not H.isEmpty()
    v = a vertex with no outgoing edges
    v.number = n
    n = n - 1
    H.remove(v)
```

Topological Sorting

Algorithm 2: Using DFS

- ◆ Simulate the algorithm by using depth-first search

Algorithm `topologicalDFS(G)`

Input: `dag G`

Output: `topological ordering of G`

```
n = G.numVertices()
for each u in G.vertices()
    u.label = UNEXPLORED
for each e in G.edges()
    e.label = UNEXPLORED
for each v in G.vertices()
    if v.label == UNEXPLORED
        topologicalDFS(G, v)
```

- ◆ $O(n+m)$ time.

Algorithm `topologicalDFS(G, v)`

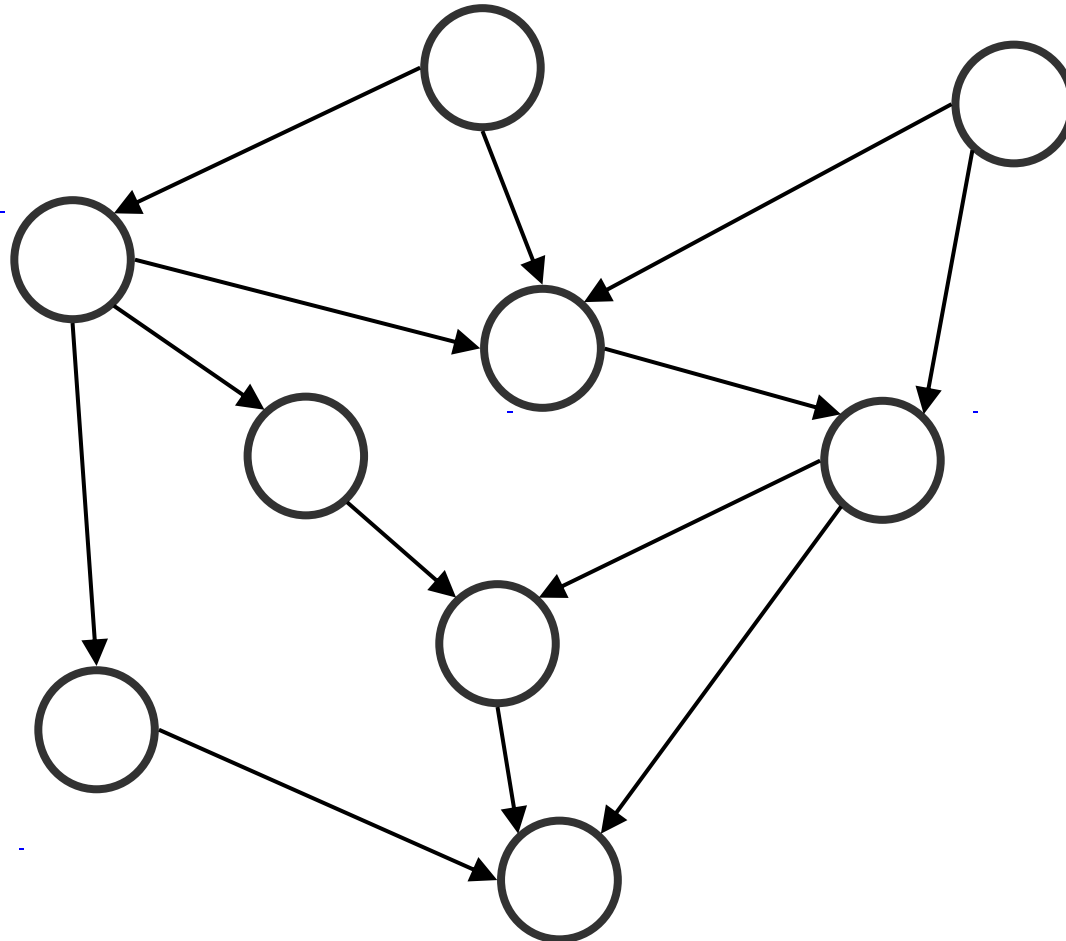
Input: `graph G` and a start vertex `v`

Output: `labeling of the vertices of G in the connected component of v`

```
v.label = VISITED
for each e in G.incidentEdges(v)
    if e.label == UNEXPLORED
        w = opposite(v,e)
        if w.label == UNEXPLORED
            e.label = DISCOVERY
            topologicalDFS(G, w)
        else
            e.label = BACK
v.number = n
n = n - 1
```

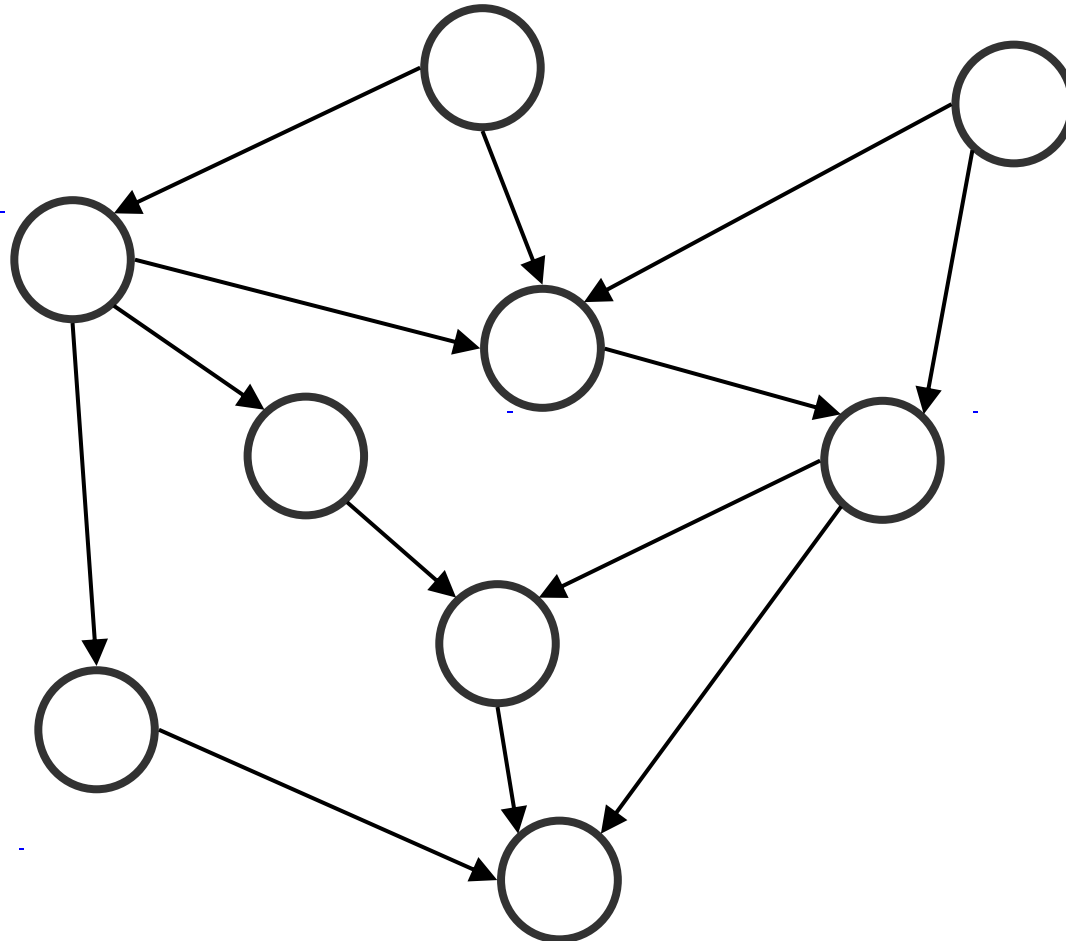
Topological Sorting 2: Using DFS

Example (Many Possible Solutions)



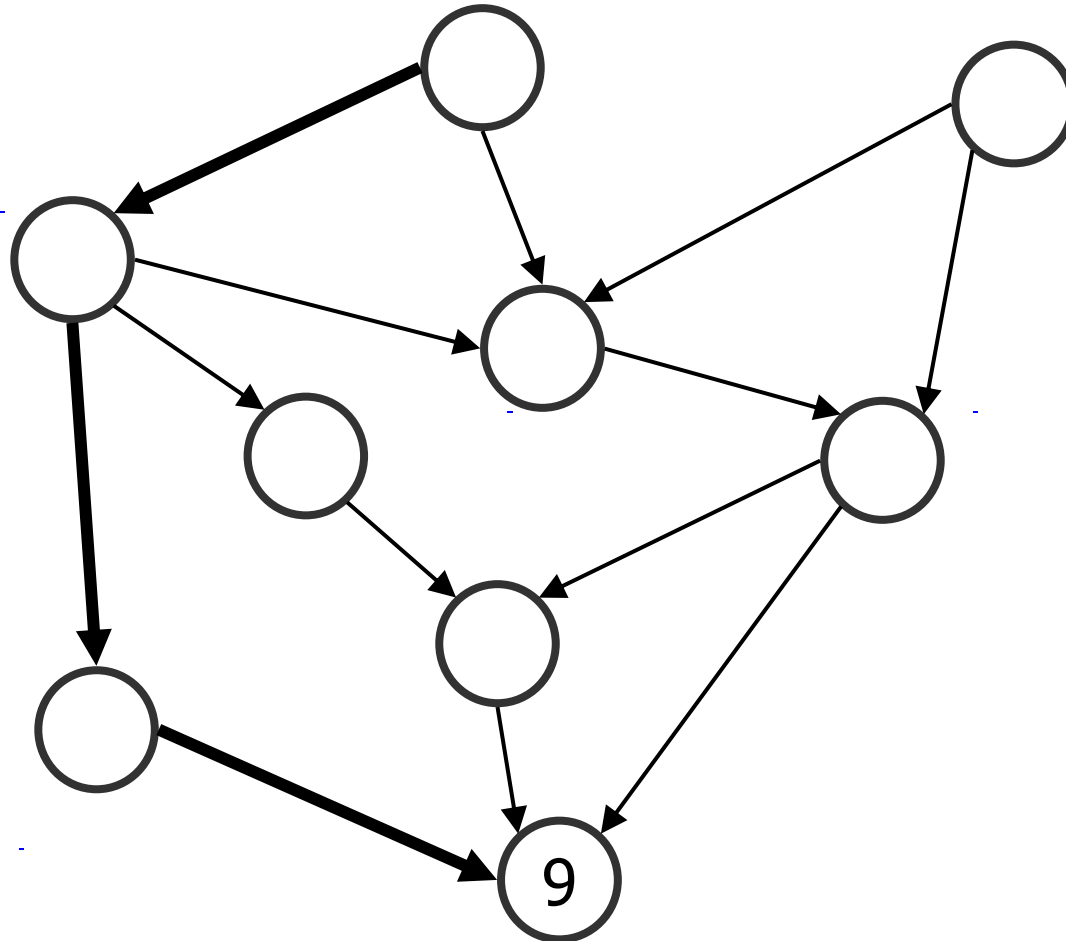
Topological Sorting 2: Using DFS

Example (Many Possible Solutions)

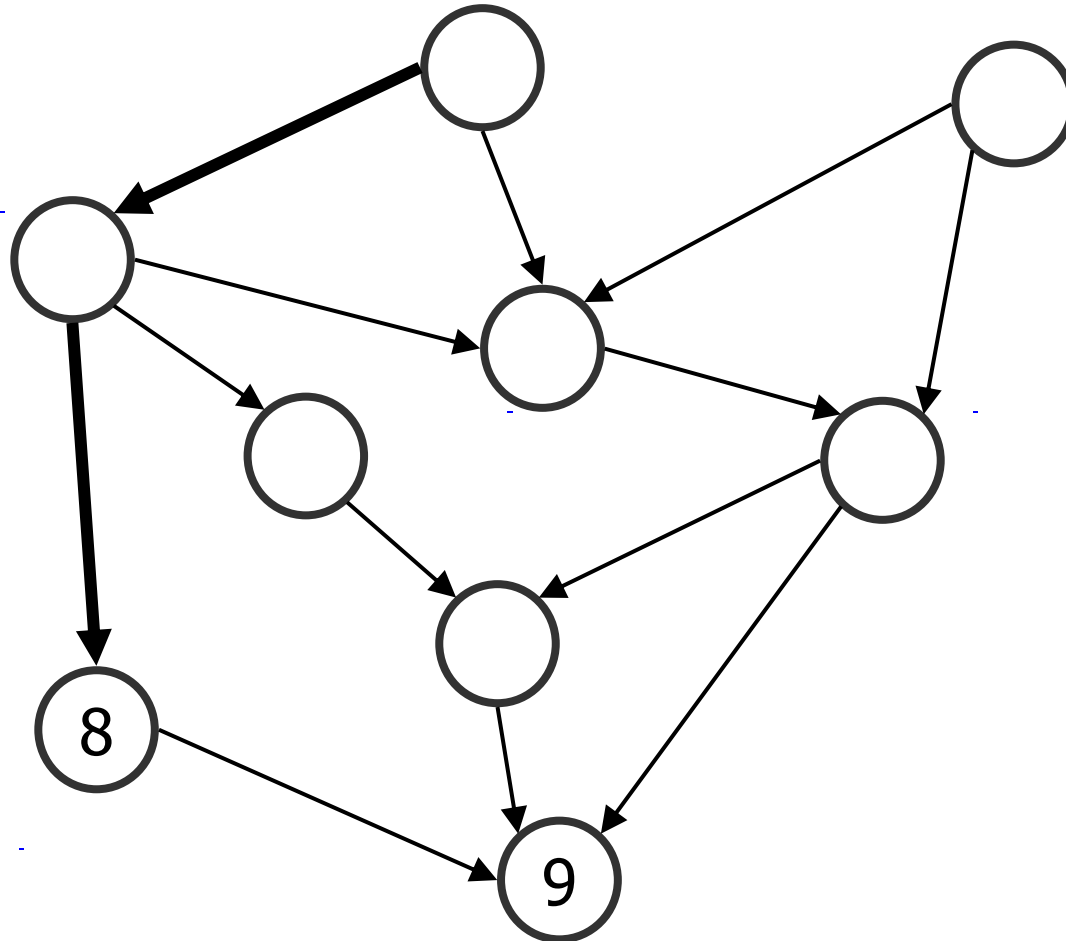


Topological Sorting 2: Using DFS

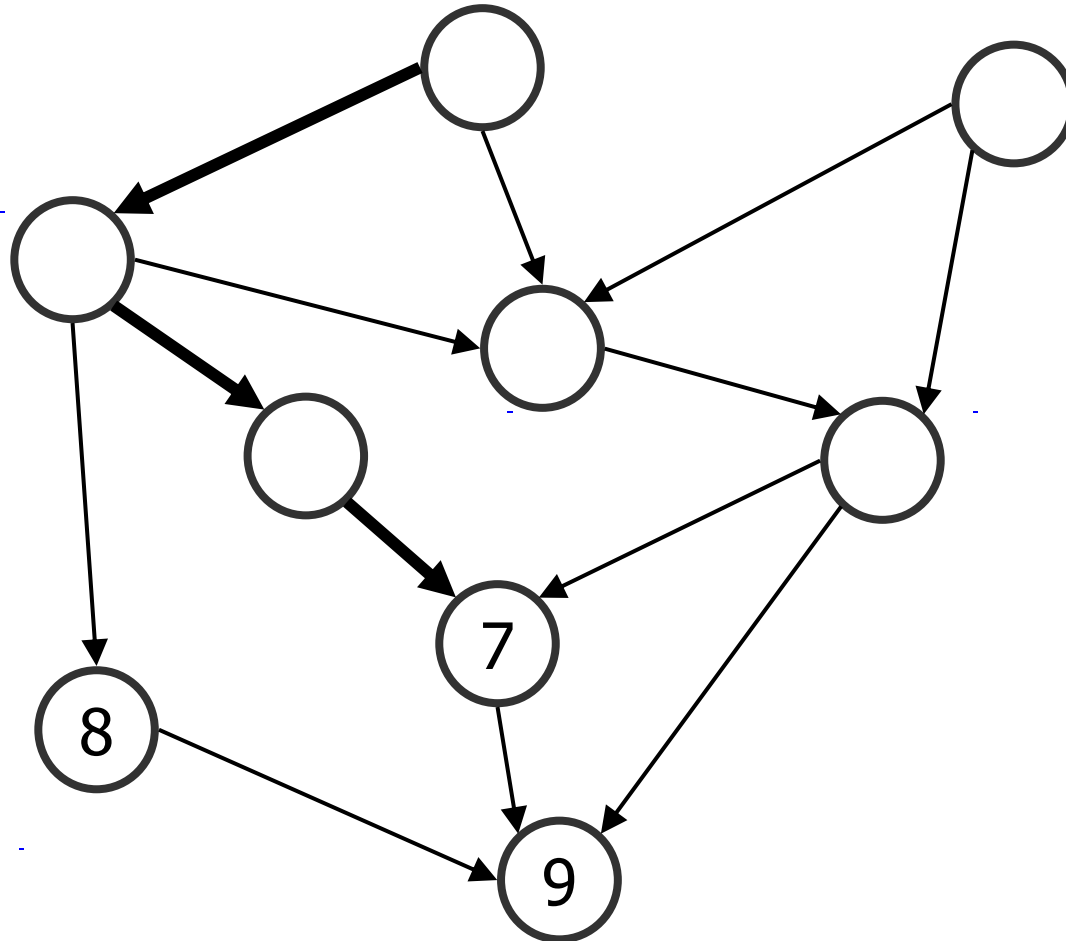
Example (Start from Top Vertex)



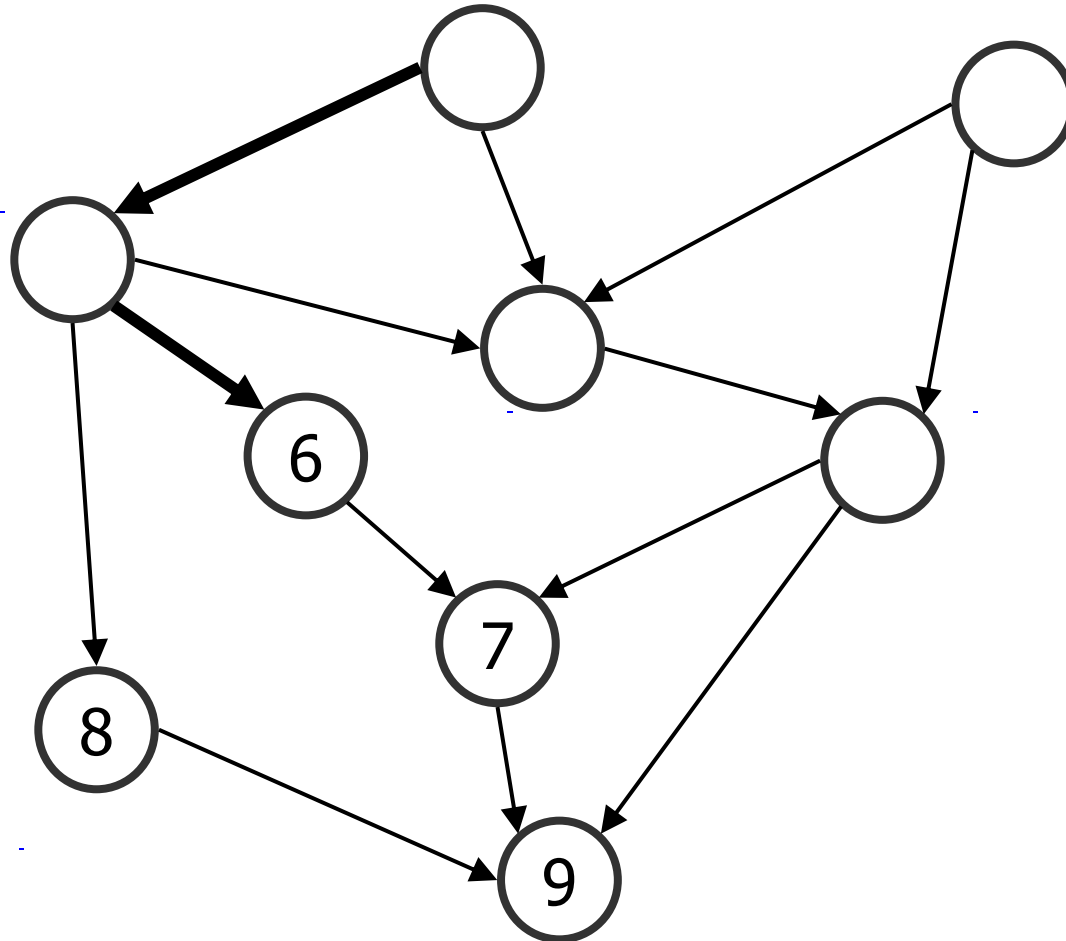
Topological Sorting 2: Using DFS Example



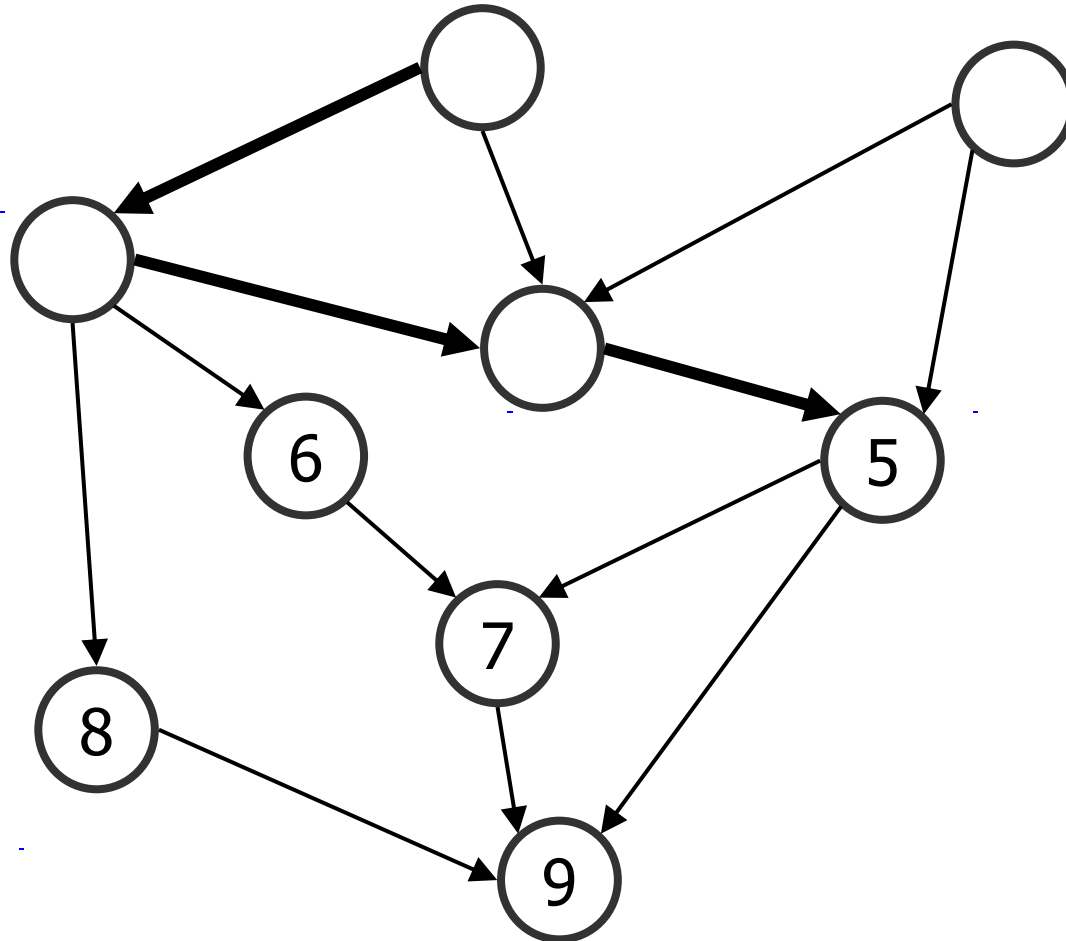
Topological Sorting 2: Using DFS Example



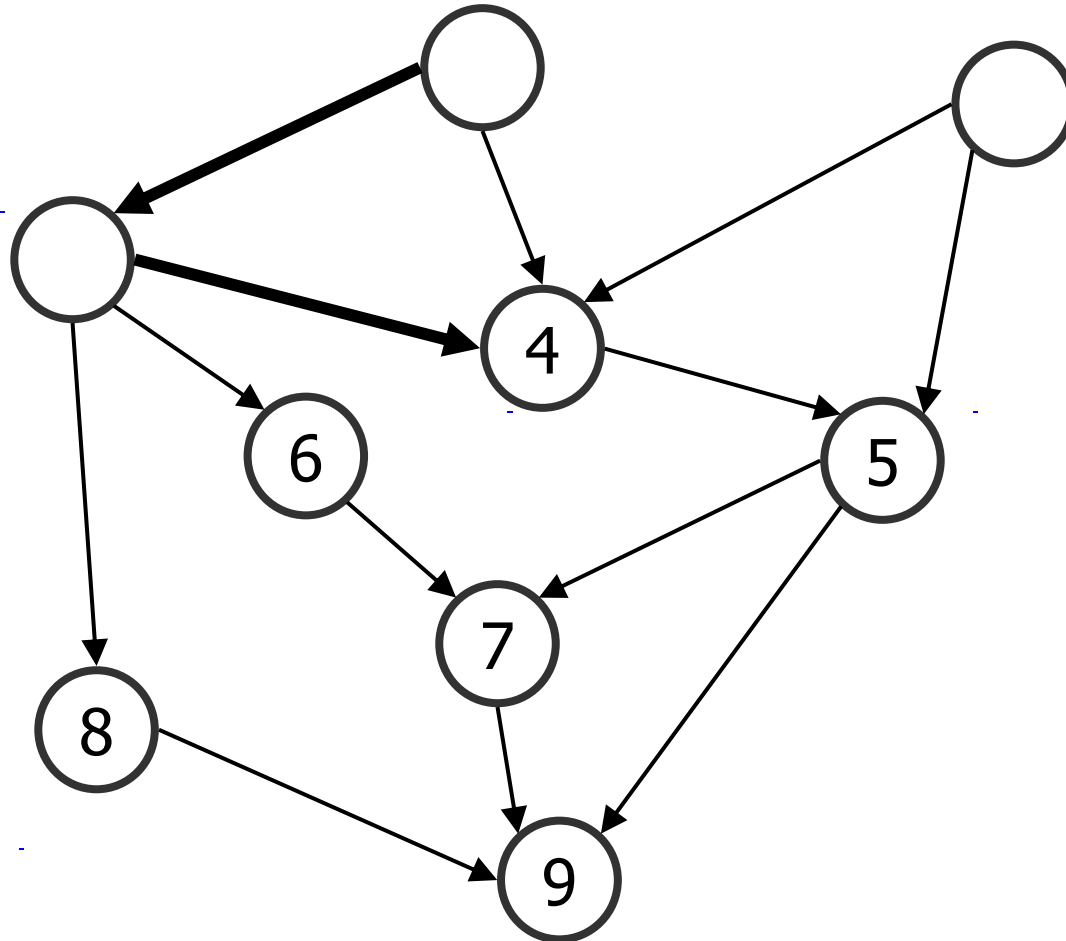
Topological Sorting 2: Using DFS Example



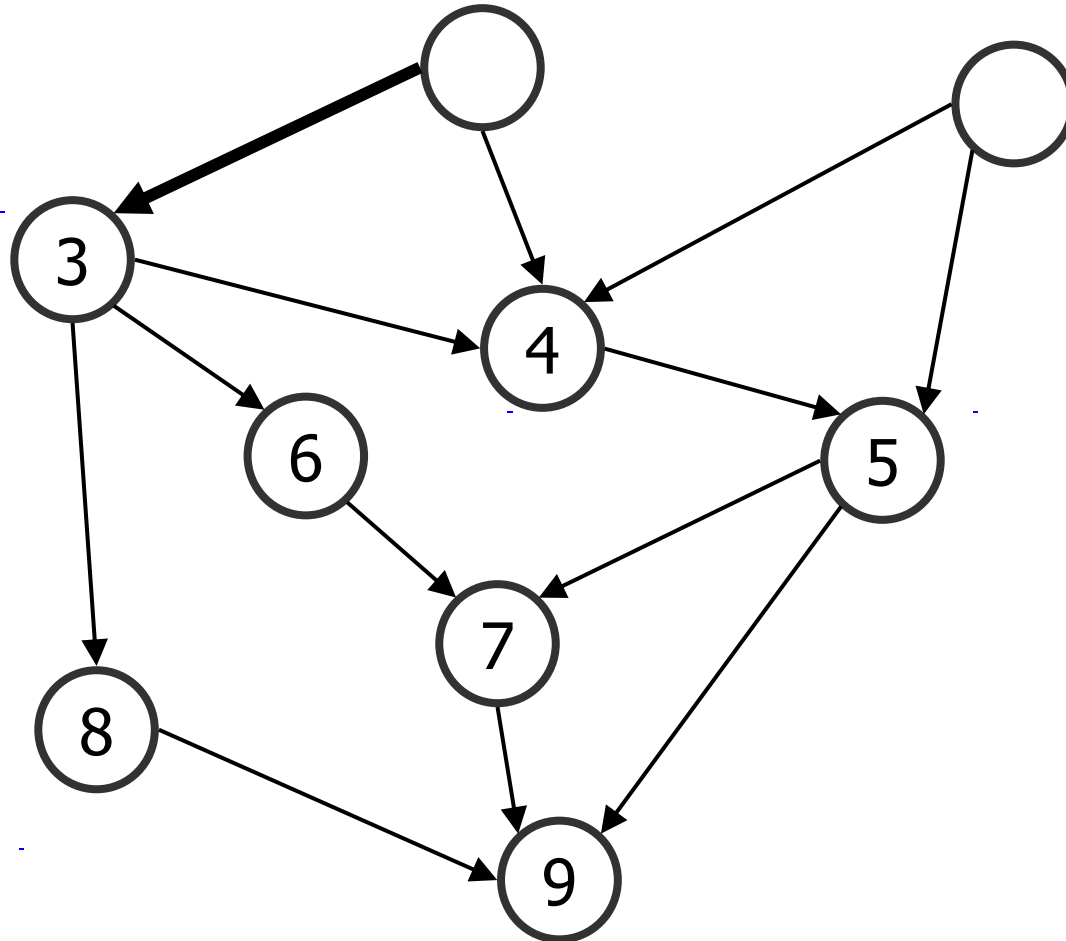
Topological Sorting 2: Using DFS Example



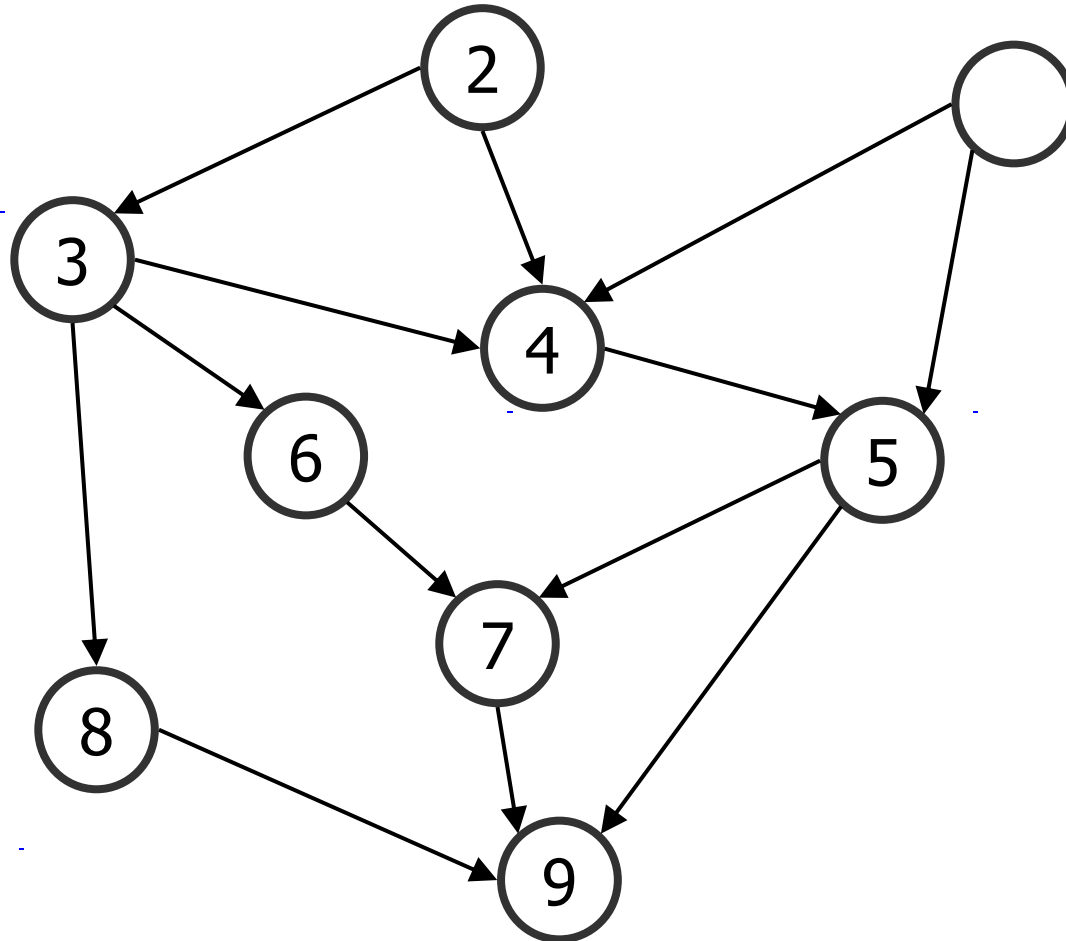
Topological Sorting 2: Using DFS Example



Topological Sorting 2: Using DFS Example



Topological Sorting 2: Using DFS Example



Topological Sorting 2: Using DFS Example

